

GUIDE SOUTENANCE — TECHNOLOGIES PAR MODULE

DR-DOCSCOL — Comment c'est fait et quoi dire au jury

Version : 1.0 — Juin 2026

Prérequis : lire d'abord docs/cahier_implementation.md (justifications globales)

Objectif : préparer le langage oral module par module, **sans répéter** les mêmes explications techniques.

Comment utiliser ce guide

- Blocs technologiques (§1)** — mémoriser une fois ; réutiliser pour plusieurs fonctionnalités.
- Modules (§2 à §4)** — pour chaque fonctionnalité : **comment c'est fait** + **blocs utilisés** + **phrase jury**.
- Si le jury demande **« pourquoi cette techno ? »** → renvoyer au **cahier d'implémentation §4**.

1. BLOCS TECHNOLOGIQUES RÉUTILISABLES

Ne pas réexpliquer ces blocs à chaque question : dire **« comme pour les autres écrans du module, on utilise le bloc B2 + B4 »**.

Bloc	Technologies	Rôle	Où dans le code
B1	Next.js Server Components	Afficher des données serveur sans exposer la BDD au client	Pages page.tsx (dashboard, admin, centre)
B2	Server Actions + Zod	Soumettre formulaires, valider, écrire en BDD	app/**/actions.ts
B3	Route Handlers REST + Zod	API HTTP (public, mobile, tests curl)	app/api/**/route.ts
B4	Prisma + PostgreSQL	Lire/écrire entités métier typées	lib/prisma.ts, services lib/*
B5	Supabase Auth + middleware	Session, protection routes, rôles	middleware.ts, lib/auth.ts
B6	react-hook-form + Tailwind + Radix	Formulaires UI accessibles	components/, pages auth/dashboard
B7	Services métier lib/*	Règles OBC/DECC centralisées	document-routing.ts, appointment-service.ts, etc.
B8	Nodemailer + notification-service	Email + notification in-app	lib/mail-service.ts, lib/notification-service.ts
B9	Supabase Storage	Upload pièces duplicata	lib/duplicata-storage.ts
B10	Parser CSV maison	Imports admin A et B	lib/csv-import-parser.ts

2. MODULE ÉLÈVE

Vue d'ensemble

Dossiers	app/dashboard/, app/consultation/, app/auth/register, components/dashboard/
----------	--

2.1 — Landing + consultation rapide (QR / matricule)

Pages	/ (landing), /consultation
Fichiers clés	components/landing/landing-page.tsx, consultation-access.tsx, lib/public-consultation.ts, app/api/public/consultation/route.ts
Blocs	B1 (affichage), B3 (API POST consultation), B4 (lookup Prisma), rate-limit (lib/rate-limit.ts)
Spécificité	Pas de session : endpoint public protégé par rate-limit ; QR via lib qrcode

Comment c'est fait

1. Landing affiche section « Consultation rapide » avec QR encodant NEXT_PUBLIC_SITE_URL/consultation.
2. L'utilisateur saisit son matricule → POST /api/public/consultation.
3. lib/public-consultation.ts cherche l'élève ; si compte activé → statuts documents (sans duplicatas).

Alternative possible

Alternative	Pourquoi non retenue
Page PHP statique + AJAX jQuery	Pas de typage, deux stacks ; moins sécurisé
Firebase Realtime DB pour statuts	Modèle relationnel déjà en Postgres ; duplication données
OAuth obligatoire pour consulter	CDC exige consultation sans compte (F-36)

Phrase jury : « La consultation rapide utilise une API REST dédiée et un rate-limit, car c'est un endpoint public sans authentification — on a séparé ça des Server Actions réservées aux utilisateurs connectés. »*

2.2 — Activation et connexion élève

Pages	/auth/register, /auth/login
Fichiers clés	app/auth/actions.ts, lib/auth.ts
Blocs	B2, B5, B4, B6

Comment c'est fait

1. **Activation** : vérif matricule + email en Prisma → création Supabase Auth → liaison authUserId.
2. **Connexion** : matricule + email + mot de passe → contrôle rôle ELEVE **avant** Supabase →

session cookie SSR.

Alternative possible

Alternative	Pourquoi non retenue
Connexion email seul	Métier OBC exige le matricule comme identifiant officiel
Mot de passe stocké Prisma	Faibles sécurité ; Supabase gère le hash

Phrase jury : *« On vérifie d'abord le matricule et le rôle en base métier, puis on délègue l'authentification à Supabase — double barrière. »*

2.3 — Mes documents (dashboard)

Page	/dashboard/documents
Fichiers clés	app/dashboard/documents/page.tsx, lib/document-routing.ts
Blocs	B1, B4, B7, B5

Comment c'est fait

- Server Component charge documents + examens validés.
- `resolveDocumentRoute()` calcule lieu de retrait, RDV requis, actions possibles (OBC/DECC).

Alternative possible

Alternative	Pourquoi non retenue
Règles en dur dans chaque page	Duplication ; erreurs si règle change (ex. Bac → antenne)
Client-side fetch API REST	Waterfall requêtes ; SSR plus rapide première peinture

Phrase jury : *« Toute la logique de routage OBC/DECC est centralisée dans `document-routing.ts` — l'écran ne fait qu'afficher le résultat. »*

2.4 — Demande relevé / diplôme

Fichiers clés	app/dashboard/actions.ts → <code>registerSchoolDocumentRequest()</code>
Blocs	B2, B4, B7, B8

Comment c'est fait

- Server Action vérifie examen validé, absence demande existante → crée/màj `DocumentAcademique PAS_DISPONIBLE` → notification.

Phrase jury : *« Même pattern que le duplicata : Server Action + service métier + notification — cohérence du module élève. »*

2.5 — Duplicata (formulaire + pièces + paiement)

Pages	/dashboard/documents (formulaire), /dashboard/payments
Fichiers clés	app/dashboard/actions.ts, lib/duplicata-service.ts, lib/duplicata-storage.ts
Blocs	B2, B4, B7, B9, B6, B8

Comment c'est fait

1. Formulaire 4 pièces → upload **Supabase Storage** (B9).
2. Barème automatique 10 000 / 15 000 FCFA → transaction Prisma (Duplicata, Paiement, Recu).
3. Paiement **simulé** MVP (pas d'API Mobile Money réelle).

Alternative possible

Alternative	Pourquoi non retenue
Stripe / PayPal	Peu utilisé au Cameroun pour ce type de service public
Orange Money API immédiate	Délai intégration + certification ; simulé pour soutenance
Pièces en local Vercel	Fichiers perdus au redeploy

Phrase jury : *« Les pièces vont dans Supabase Storage car Vercel n'a pas de disque persistant ; le paiement est simulé en attendant l'API Mobile Money certifiée. »*

2.6 — Rendez-vous de retrait

Page	/dashboard/rendez-vous
Fichiers clés	lib/appointment-service.ts, app/api/appointments/slots/route.ts
Blocs	B2, B3 (créneaux), B4, B7, B8

Comment c'est fait

- getAvailableSlots() : quota journalier, week-ends, jours fériés, RDV existants.
- Réservation → statut **PLANIFIE** → visible chez l'agent (module Agent).

Alternative possible

Alternative	Pourquoi non retenue
Calendly externe	Pas intégré au statut document Prisma ; pas de traçabilité admin
Créneaux fixes en dur	Admin doit paramétrer quota (F-30)

Phrase jury : *« Les créneaux passent par une API GET pour pouvoir rafraîchir le calendrier sans recharger toute la page. »*

3. MODULE ADMINISTRATEUR OBC / DECC

Vue d'ensemble

Dossiers	app/admin/, components/admin/, lib/admin-*.ts
-----------------	---

Note : Auth admin = même B5 que l'élève, pages séparées /auth/login/obc et /auth/login/decc + scope lib/admin-scope.ts.

3.1 — Connexion et périmètre admin

Pages	/auth/login/obc, /auth/login/decc
Fichiers clés	lib/admin-scope.ts, lib/auth.ts
Blocs	B5, B4

Spécificité : chaque admin limité à organismeId + antenneRegionaleId — pas d'accès national croisé OBC/DECC.

Phrase jury : *« Un admin DECC ne voit que les élèves et documents de son organisme et sa région — contrôle dans admin-scope.ts, pas seulement dans l'UI. »*

3.2 — Import élèves (Import B)

Page	/admin/students
Fichiers clés	lib/admin-student-import.ts, lib/csv-import-parser.ts
Blocs	B2, B4, B10, B7

Comment c'est fait

- Upload CSV → parsing ligne par ligne → UPSERT élève + examens + documents
PAS_DISPONIBLE → rapport erreurs par ligne.

Alternative possible

Alternative	Pourquoi non retenue
Saisie manuelle uniquement	Impossible à l'échelle (milliers d'élèves par session)
Excel + macro	Non traçable ; pas d'audit automatique
Import SQL direct	Risque sécurité ; admin non technique

Phrase jury : *« L'import CSV réutilise le même parser que l'import disponibilisation — un seul moteur de parsing, deux en-têtes métier. »*

3.3 — Import disponibilisation (Import A)

Page	/admin/import
Fichiers clés	lib/document-availability-import.ts, lib/document-status-transition.ts
Blocs	B2, B4, B10, B7, B8

Comment c'est fait

- 1 ligne CSV = 1 document → DISPONIBLE → notification élève via transition centralisée.

Phrase jury : *« Chaque changement de statut passe par document-status-transition.ts — import ou action manuelle, même chemin, même audit. »*

3.4 — Gestion documents et MAJ statut unitaire

Page	/admin/documents
Fichiers clés	app/admin/actions.ts, lib/document-status-transition.ts
Blocs	B1, B2, B4, B7, B8

Spécificité : retrait antenne (RETIRE) réservé à l'admin ; pas à l'agent centre.

3.5 — Validation duplicata

Fichiers clés	lib/duplicata-service.ts, app/admin/actions.ts
Blocs	B2, B4, B7, B9 (lecture pièces Storage)

Phrase jury : *« L'admin valide les pièces stockées dans Supabase Storage ; une fois le dossier VALIDEE, l'import A peut disponibiliser le duplicata. »*

3.6 — Quota RDV et audit

Pages	/admin/rdv-disponibilites, /admin/audit-logs
Blocs	B2, B1, B4

Note soutenance : fonctionnalités **paramétrage** — même stack que le reste admin ; pas de techno supplémentaire.

4. MODULE AGENT CENTRE D'EXAMEN

Vue d'ensemble

Dossiers	app/centre-examen/, lib/centre-examen-service.ts
----------	--

4.1 — Connexion agent

Page	/auth/login/centre-examen
Blocs	B5, B4 — identique auth élève/admin, rôle AGENT_CENTRE_EXAMEN

4.2 — Consulter les RDV transmis

Page	/centre-examen
Fichiers clés	lib/centre-examen-service.ts, app/api/centre-examen/appointments/route.ts
Blocs	B1, B3, B4, B7

Comment c'est fait

- Filtre automatique par centre d'examen de l'agent — ne voit pas duplicatas ni Bac original antenne.

Phrase jury : *« Le filtre centre est dans centre-examen-service.ts côté serveur — l'agent ne peut pas contourner en modifiant l'URL. »*

4.3 — Confirmer retrait effectué

Fichiers clés	app/api/centre-examen/confirm-withdrawal/route.ts
Blocs	B3, B4, B7, B8

Comment c'est fait

- POST API → RDV HONORE, document RETIRE, notifications, audit.

Alternative possible

Alternative	Pourquoi non retenue
Admin confirme aussi les retraits centre	CDC : centre = agent uniquement (F-29) ; séparation des rôles
Signature papier seule	Pas de traçabilité numérique

Phrase jury : *« Seul l'agent du centre confirme le retrait physique au guichet ; l'admin gère les retraits en antenne régionale. »*

5. TABLEAU SYNTHÈSE — QUOI DIRE EN 30 SECONDES PAR MODULE

Module	Technologies dominantes	Message clé jury
Élève	Next.js SSR + Server Actions + Prisma + Supabase Auth	Parcours unifié TypeScript ; consultation publique en API REST protégée
Admin	Server Actions + CSV parser + transitions statut centralisées	Imports massifs + règles métier dans lib/*, scope OBC/DECC strict
Agent	API REST + filtre centre serveur	Rôle minimal : voir RDV, confirmer retrait — pas d'accès back-office

6. QUESTIONS JURY FRÉquentes — RÉPONSES COURTES

Question	Réponse (renvoi)
Pourquoi pas Laravel / PHP ?	Cahier impl. §4.1 — stack TypeScript unifiée, SSR

	native
Pourquoi Supabase ?	§4.4 + §4.3 — Auth + Postgres + Storage même plateforme
Où sont les règles OBC/DECC ?	lib/document-routing.ts, pas dispersées dans l'UI
Paiement Mobile Money ?	Simulé MVP ; §7 cahier impl. — API réelle en évolution
Sécurité consultation publique ?	Rate-limit + pas de fuite si compte non activé
Différence Server Action vs API ?	Actions = formulaires internes ; API = public et agent

Fin du guide — DR-DOCSCOL v1.0